

Subject: CS987 – Advanced Critical Infrastructure

Name: P. Sivachandran (Roll number: 241560049)

Date: 6th June 2026

Assignment 2: Reconnaissance and Exploitation of a Conpot Based OT/SCADA Target

Part A – Host Discovery and Service Enumeration

Objective of assessment is to perform controlled penetration test against simulated environment (OT/SCADA) implemented using Conpot. Accordingly discover “Target system”, “Enumeration of exposed services”, identification of industrial protocols, and evaluating potential risks associated with those services.

Conpot target discovery

Start with host discovery:

```
nmap -sn 192.168.56.0/24
```

Following three hosts were discovered on the subnet

192.168.56.1,

192.168.56.100 &

192.168.56.101

```
sivachandran@VirtualBox:~/conpot$ sudo nmap -sn 192.168.56.0/24
[sudo] password for sivachandran:
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 13:27 JST
Nmap scan report for 192.168.56.1
Host is up (0.00021s latency).
MAC Address: 0A:00:27:00:00:10 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.000096s latency).
MAC Address: 08:00:27:81:2C:9B (Oracle VirtualBox virtual NIC)
Nmap scan report for VirtualBox (192.168.56.101)
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.04 seconds
```

here 192.168.56.101 is identified as OT target, since exposed industrial protocol ports

associated with PLCs and SCADA systems.

Step – 2: TCP Service Enumeration

Then perform service enumeration:

```
nmap -sS -sV -O -p- <target-ip>
```

```
sivachandran@VirtualBox:~/conpot$ sudo nmap -sS -sV -O -p- 192.168.56.101
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 13:38 JST
Nmap scan report for VirtualBox (192.168.56.101)
Host is up (0.000045s latency).
Not shown: 65530 closed ports
PORT      STATE      SERVICE      VERSION
21/tcp    filtered  ftp
80/tcp    filtered  http
102/tcp   filtered  iso-tsap
502/tcp   filtered  mbap
44818/tcp filtered  EtherNetIP-2
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 3.81 seconds
```

FTP (TCP/21): Used for Firmware upload, engineering files, and configuration backups. Exposure may allow attackers to get sensitive operational data

HTTP (TCP/80): Web-based interface, such interfaces often expose device status, operational data and configuration settings

Siemens S7 (TP/102): Port 102 associated with Siemens S7 communication, used by engineering workstations to communication with Siemens PLCs.

Modbus/TCP (TCP/502): Port 502 is most widely deployed in industrial protocols (which may help to control Pump, valves, motors, alarms, counters: Reading coils, Input status, holding registers, process variables, potential write operations)

Ethernet/IP-2 (TCP/44818): Exposure to reveal controller tags, process information and device metadata.

UDP enumeration

Consider UDP enumeration where appropriate:

```
nmap -sU --top-ports 50 <target-ip>
```

```
sivachandran@VirtualBox:~/conpot$ sudo nmap -sU --top-ports 50 192.168.56.101
[sudo] password for sivachandran:
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 17:16 JST
Nmap scan report for VirtualBox (192.168.56.101)
Host is up (0.0000040s latency).
Not shown: 47 closed ports
PORT      STATE      SERVICE
69/udp    open|filtered tftp
161/udp    open|filtered snmp
5353/udp   open|filtered zeroconf

Nmap done: 1 IP address (1 host up) scanned in 1.28 seconds
sivachandran@VirtualBox:~/conpot$
```

TFTP (UDP/69): Used for Firmware update & configuration transfer (exposure to allow unauthorized access to configuration files)

SNMP (UDP/161): Used to get device name, asset info., system description, Firewall details, network info etc. Which could assist attacker reconnaissance

mDNS (UDP/5353): Used to discover functionality to capture hostnames etc.

Part – B: Web/HMI Reconnaissance

Objective is to identify and assess any web-based interface exposed by OT/SCADA target. Revealing of following operational data helps attackers understanding the industrial environment – Human Machine Interfaces, Device status dashboards, Maintenance portals, Engineering workstations Configuration panels.

Target discovered in the isolated network 192.168.56.0/24 and identified Conpot-based industrial control system simulation.

HTTP header inspection

```
curl -i http://<target-ip>/  
nikto -h http://<target-ip>/  
gobuster dir -u http://<target-ip>/ -w <wordlist>
```

```
sivachandran@VirtualBox:~/conpot$ curl -i http://192.168.56.101  
HTTP/1.1 302 Found  
Date: Sun, 31 May 2026 09:00:49 GMT  
Content-Type: text/html  
Location: /index.html  
Content-Length: 0
```

Here server automatically redirects users from the root page (/index.html), indicates the presence of web application rather than default web server page.

```
sivachandran@VirtualBox:~/conpot$ curl -v http://192.168.56.101  
* Trying 192.168.56.101:80...  
* Connected to 192.168.56.101 (192.168.56.101) port 80 (#0)  
> GET / HTTP/1.1  
> Host: 192.168.56.101  
> User-Agent: curl/7.81.0  
> Accept: */*  
>  
* Mark bundle as not supporting multiuse  
< HTTP/1.1 302 Found  
< Date: Sun, 31 May 2026 09:00:49 GMT  
< Content-Type: text/html  
< Location: /index.html  
< Content-Length: 0  
<  
* Connection #0 to host 192.168.56.101 left intact  
sivachandran@VirtualBox:~/conpot$
```

No explicit server banner disclosed, and attacker could not identify immediately

(Operating System, Web-server software, Server version)

Directory enumeration

No any administrative or maintenance directories could identify using common wordlist. (it looks restricted Conpot template designed to expose only limited functionality and minimal web-interace)

```
^Csivachandran@VirtualBox:~/conpot$ gobuster -m dir -u http://192.168.56.101/ -w /usr/share/dirb/wordlists/common.txt
=====
Gobuster v2.0.1                OJ Reeves (@TheColonial)
=====
[+] Mode       : dir
[+] Url/Domain  : http://192.168.56.101/
[+] Threads    : 10
[+] Wordlist    : /usr/share/dirb/wordlists/common.txt
[+] Status codes : 200,204,301,302,307,403
[+] Timeout    : 10s
=====
2026/05/31 18:07:13 Starting gobuster
=====
/index.htm (Status: 200)
/index.html (Status: 200)
=====
2026/05/31 18:07:32 Finished
=====
sivachandran@VirtualBox:~/conpot$
```

Below Nikto command helps to identify weak HTTP security controls.

```
sivachandran@VirtualBox:~/conpot$ nikto -h http://192.168.56.101
- Nikto v2.1.5
-----
+ Target IP:      192.168.56.101
+ Target Hostname: 192.168.56.101
+ Target Port:    80
+ Start Time:     2026-05-31 18:06:06 (GMT9)
-----
+ Server: No banner retrieved
+ The anti-clickjacking X-Frame-Options header is not present.
+ Root page / redirects to: /index.html
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Cookie path created without the httponly flag
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE
+ OSVDB-877: HTTP TRACE method is active, suggesting the host is vulnerable to XST
```

Security understanding is not severe isolated lab environment, it demonstrates weak web-service hardening.

Missing Http Only Cookie flag: If cross-site scripting vulnerability exists, cookies are more easily accessed by client-side scripts.

In addition to it, below command and response reveals significant vendor information (vendor/brand name – Siemens, product family - SIMATIC, PLC series (S7-200), communication module – CP 443-1 EX40)
curl http://192.168.56.101/maintenance/flag.txt

Looks Conpot template intentionally expose to emulate real industrial environment.

This web-interface leaks information and attacker could research and understand the industrial environment (industrial protocols in use, vendor-specific vulnerabilities, targeted phishing or social engineering attacks, build accurate attacks against workstations and PLCs)

In this way web content can provide valuable reconnaissance info.

Regarding default credential test, there is not login page discovered during reconnaissance, therefore

No authentication portal was identified

No default credentials were tested

```
sivachandran@VirtualBox:~/conpot$ curl http://192.168.56.101/maintenance/flag.tx
t
<HTML>

  <HEAD>
    <TITLE>Not found - Siemens, SIMATIC, S7-200</TITLE>
  </HEAD>

  <BODY>
    <h2>CP 443-1 EX40</h2>
    <hr>
    &nbsp;<br>
    This resource could not be found.<br>
  </BODY>

</HTML>sivachandran@VirtualBox:~/conpot$
```

In real OT environment, similar web interfaces may provide the following details:
Process status, Alarm data, Engineering settings, Device diagnostics, Firmware management, Configuration capabilities.

Accordingly, this web service resembles device status or maintenance portal, such reconnaissance may help to support attack against industrial control systems.

Part – C: ICS Protocol Fingerprinting

Objective is to identify Industrial Control System (ICS) and Operational Technology (OT) protocols exposed by the target and determine what information or functionality they provide.

In general ICS protocols expose process data, device status information, and control functions which could directly affect industrial operations.

Identification of Modbus/SNMP/Other OT services

```
nmap -sV -p 502 <target-ip>
```

If available, try:

```
nmap --script modbus-discover -p 502 <target-ip>
```

This result shows presence of multiple protocols commonly associated with industrial environments. “mbap” refers to Modbus Application Protocol

TCP port 502 identified as Modbus/TCP, widely used industrial communication protocol used between SCADA systems HMIs and PLCs. **TCP port 44818** identified as EtherNet/IP.

```
sivachandran@VirtualBox:~/conpot$ sudo nmap -sS -sV -O -p- 192.168.56.101
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 13:38 JST
Nmap scan report for VirtualBox (192.168.56.101)
Host is up (0.000045s latency).
Not shown: 65530 closed ports
PORT      STATE      SERVICE      VERSION
21/tcp    filtered  ftp
80/tcp    filtered  http
102/tcp   filtered  iso-tsap
502/tcp   filtered  mbap
44818/tcp filtered  EtherNetIP-2
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 3.81 seconds
```

```

sivachandran@VirtualBox:~/conpot$ sudo nmap -sV -p 502 192.168.56.101
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 22:05 JST
Nmap scan report for VirtualBox (192.168.56.101)
Host is up.

PORT      STATE      SERVICE VERSION
502/tcp   filtered  mbap

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.27 seconds
sivachandran@VirtualBox:~/conpot$ sudo nmap --script modbus-discover -p 502 192.168.56.101
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 22:07 JST
Nmap scan report for VirtualBox (192.168.56.101)
Host is up.

PORT      STATE      SERVICE
502/tcp   filtered  mbap

Nmap done: 1 IP address (1 host up) scanned in 2.17 seconds
sivachandran@VirtualBox:~/conpot$

```

```

sivachandran@VirtualBox:~/conpot$ sudo nmap -sU --top-ports 50 192.168.56.101
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-31 21:25 JST
Nmap scan report for VirtualBox (192.168.56.101)
Host is up (0.000015s latency).
Not shown: 47 closed ports
PORT      STATE      SERVICE
69/udp    open|filtered  tftp
161/udp    open|filtered  snmp
5353/udp   open|filtered  zeroconf

Nmap done: 1 IP address (1 host up) scanned in 1.36 seconds
sivachandran@VirtualBox:~/conpot$

```

Since “Port 502” is open,

SNMP could reveal device info. and network management information which will guide attackers in mapping industrial environment

Modbus communication expose operational process data through register and coil reads

Clear port-to-protocol mapping:

TCP/502 – Modbus/TCP

TCP/102 – Siemens S7/ISO-TSAP

TCP/44818 – EtherNet/IP

UDP/161 - SNMP

Protocol-specific tools or scripts:

While executed below code to read_holding_registers(...), got the attached feedback


```
Open  ▾  [⌂]  ModbusReadOperations_5.py
~/.conpot

1 from pymodbus.client import ModbusTcpClient
2
3 client = ModbusTcpClient("192.168.56.101", port=502)
4
5 if client.connect():
6
7     result = client.read_holding_registers(
8         address=0,
9         count=10,
10        device_id=1
11    )
12    print(result)
13
14    client.close()
15
16 else:
17     print("Connection failed")
```

```
sivachandran@VirtualBox:~/conpot$ python3 ModbusReadOperations_5.py
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
sivachandran@VirtualBox:~/conpot$
```

```
sivachandran@VirtualBox:~/conpot$ nc -vz 127.0.0.1 502
Connection to 127.0.0.1 502 port [tcp/*] succeeded!
sivachandran@VirtualBox:~/conpot$
```

```

sivachandran@VirtualBox:~/conpot$ snmpwalk -v2c -c public 192.168.56.101
iso.3.6.1.2.1.1.1.0 = STRING: "Siemens, SIMATIC, S7-200"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.20408
iso.3.6.1.2.1.1.3.0 = Timeticks: (379) 0:00:03.79
iso.3.6.1.2.1.1.4.0 = STRING: "Siemens AG"
iso.3.6.1.2.1.1.5.0 = STRING: "CP 443-1 EX40"
iso.3.6.1.2.1.1.6.0 = STRING: "Venus"
iso.3.6.1.2.1.1.7.0 = INTEGER: 72
iso.3.6.1.2.1.1.8.0 = Timeticks: (0) 0:00:00.00
iso.3.6.1.2.1.11.1.0 = Counter32: 11
iso.3.6.1.2.1.11.2.0 = Counter32: 0
iso.3.6.1.2.1.11.3.0 = Counter32: 0
iso.3.6.1.2.1.11.4.0 = Counter32: 0
iso.3.6.1.2.1.11.5.0 = Counter32: 0
iso.3.6.1.2.1.11.6.0 = Counter32: 0
iso.3.6.1.2.1.11.8.0 = Counter32: 0
iso.3.6.1.2.1.11.9.0 = Counter32: 0
iso.3.6.1.2.1.11.10.0 = Counter32: 0
iso.3.6.1.2.1.11.11.0 = Counter32: 0
iso.3.6.1.2.1.11.12.0 = Counter32: 0
iso.3.6.1.2.1.11.13.0 = Counter32: 0
iso.3.6.1.2.1.11.14.0 = Counter32: 0
iso.3.6.1.2.1.11.15.0 = Counter32: 0
iso.3.6.1.2.1.11.16.0 = Counter32: 0
iso.3.6.1.2.1.11.17.0 = Counter32: 0
iso.3.6.1.2.1.11.18.0 = Counter32: 0
iso.3.6.1.2.1.11.19.0 = Counter32: 0
iso.3.6.1.2.1.11.20.0 = Counter32: 0
iso.3.6.1.2.1.11.21.0 = Counter32: 0
iso.3.6.1.2.1.11.22.0 = Counter32: 0
iso.3.6.1.2.1.11.24.0 = Counter32: 0
iso.3.6.1.2.1.11.27.0 = Counter32: 0
iso.3.6.1.2.1.11.28.0 = Counter32: 0
iso.3.6.1.2.1.11.29.0 = Counter32: 0
iso.3.6.1.2.1.11.30.0 = INTEGER: 1
iso.3.6.1.2.1.11.31.0 = Counter32: 0
iso.3.6.1.2.1.11.32.0 = Counter32: 0
iso.3.6.1.2.1.11.32.0 = No more variables left in this MIB View (It is past the end of the MIB tree)
sivachandran@VirtualBox:~/conpot$

```

```

sivachandran@VirtualBox:~/conpot$ sudo tcpdump -i lo port 502 -w modbus_capture.pcap
[sudo] password for sivachandran:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
144 packets captured
288 packets received by filter
0 packets dropped by kernel
^Csivachandran@VirtualBox:~/conpot$

```

Understanding of why exposed OT protocols matter:

Port 80 is open: Web service could disclose Siemens SIMATIC references for attacker to identify the device type

Port 502 is open: Exposure of Modbus/TCP is critical because it may provide direct access to industrial process data and control functions (Read process values, coil status, alarms, modify registers/coils)

“In the case of IT service leak may help to acquire information for attacker, but OT protocol may expose process state or control primitives.”

Example, Modbus - Register may be mapped with physical end unit device (water tank level or Pressure of pump or Temperature monitor), which could be accessed by attacker and set any Register values (stop the pump, open valve, disable alarm, change setpoint).

Exposure of Modbus/TCP represents critical risk than ordinary information leakage since it could share direct access to process state and control functions.

Part – D: Modbus Register and Coil Interaction

Objective is to interact with the exposed Modbus/TCP service (TCP port 502) in Conpot OT/SCADA target and identify the feasibilities to read process data through standard Modbus function codes.

Target system identified from attacker machine during host discovery and service enumeration at IP address 192.168.56.101

```
^Csivachandran@VirtualBox:~/conpot$ nc -vz 192.168.56.101 502
Connection to 192.168.56.101 502 port [tcp/*] succeeded!
sivachandran@VirtualBox:~/conpot$
```

Attempt:

```
read coils
read discrete inputs
read holding registers
read input registers
```

For each attempt, record:

Attempt controlled Modbus interaction with the target.
Example read-only interaction:

```
from pymodbus.client import ModbusTcpClient

target = "192.168.56.20"
client = ModbusTcpClient(target, port=502)

client.connect()

rr = client.read_holding_registers(address=0, count=10, slave=1)
print(rr)

client.close()
```

Depending on your installed version of pymodbus, the argument may be:

slave=1

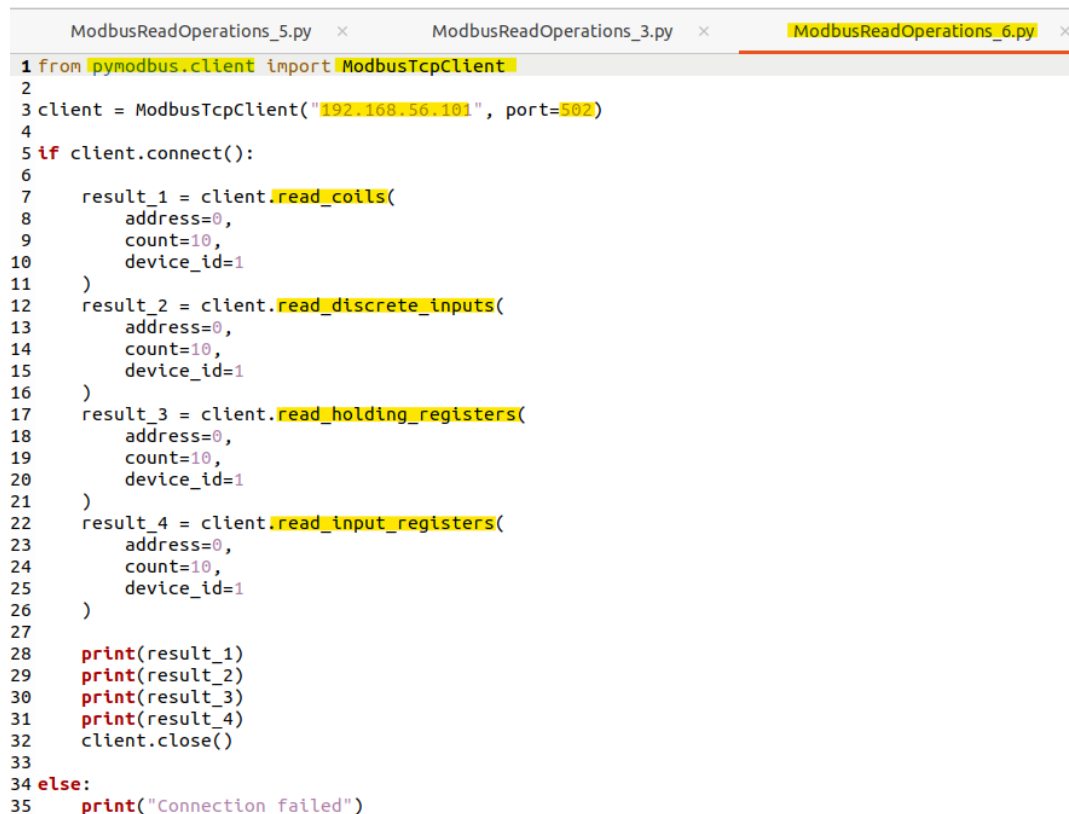
or:

unit=1

Resolve such tooling issues and document what worked.

Modbus Read operations:

Below captured “ModbusReadOperations_6.py” script using “pymodbus” library to perform below four Modbus read functions (Read Coils, Discrete input, Holding registers, Input Registers)



The screenshot shows a code editor with three tabs: 'ModbusReadOperations_5.py', 'ModbusReadOperations_3.py', and 'ModbusReadOperations_6.py'. The 'ModbusReadOperations_6.py' tab is active and contains the following Python code:

```
1 from pymodbus.client import ModbusTcpClient
2
3 client = ModbusTcpClient("192.168.56.101", port=502)
4
5 if client.connect():
6
7     result_1 = client.read_coils(
8         address=0,
9         count=10,
10        device_id=1
11    )
12    result_2 = client.read_discrete_inputs(
13        address=0,
14        count=10,
15        device_id=1
16    )
17    result_3 = client.read_holding_registers(
18        address=0,
19        count=10,
20        device_id=1
21    )
22    result_4 = client.read_input_registers(
23        address=0,
24        count=10,
25        device_id=1
26    )
27
28    print(result_1)
29    print(result_2)
30    print(result_3)
31    print(result_4)
32    client.close()
33
34 else:
35     print("Connection failed")
```

“device_id=1” is used since version of pymodbus (3.13.0) requires “device_id”.

```
sivachandran@VirtualBox:~/conpot$ python3 ModbusReadOperations 6.py
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
sivachandran@VirtualBox:~/conpot$
```

Read_coils ()– helps to read output (of Ex. Pump status, Motor ON/OFF state, Relay output, Actuator output and etc.)

Function code **129** = Exception response to Function Code **1**

Exception code 2 (illegal data address) indicates requested coil address are not available in the Conpot Modbus configuration.

Read_discrete_inputs () – helps to read input (of Ex. Alarm indicators, Switch status, Sensor state, Limit switches and etc.)

Function code **130** = Exception response to Function Code **2**

Exception code 2 indicates that no readable discrete input exist at the requested addresses.

Read_holding_registers () – Helps holding registers (typically Ex. Process variables, Controller settings, Configuration parameters, Setpoints, Counters etc.)

Function code **131** = Exception response to Function Code **3**

Modbus server received the request but reported that the requested register addresses are invalid or not implemented.

Read_input_registers () – Helps to read input registers (of Ex. Sensor readings, Temperature values, Pressure values, Flow measurements, Process data and etc.)

Function code **132** = Exception response to Function Code **4**

The server reported that the requested input register addresses are unavailable.

Even though none of the requested Modbus objects returned valid data, important point to note, **for every request received a valid Modbus exception response from server**. It demonstrates that service is active but intentionally limits access to certain coils and registers.

Modbus/TCP service is reachable, and Target successfully processing each request.

Above test demonstrates that Modbus service is reachable and actively processing our requests. Which may help to read equipment sensitive details such as, process state information, sensor output measurements, equipment status, alarm condition, operational counters, engineering setpoints etc.

Modbus protocol implementation identifies server and found that functioning properly, but the requested addresses are not available within the Conpot template configuration.

To get the count of packets captured, used below command,

```
sivachandran@VirtualBox:~/conpot$ sudo tcpdump -i lo port 502 -w modbus_capture.pcap
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C7 packets captured
14 packets received by filter
0 packets dropped by kernel
sivachandran@VirtualBox:~/conpot$
```

Part – E: Controlled Write or Manipulation Attempt

Objective is to perform controlled Modbus write operations against conpot OT/SCADA simulator and check whether target accept the modifications to Coil & Holding registers.

Target: 192.168.56.101, Port: 502

Write_coil () – helps to perform Modbus write (but in the actual scenario, Coil “Value - True” write attempt got failed, due to illegal data address)

Function code **133** = Exception response to Function Code **5**

Exception code 2 - indicates requested **coil address does not exist or not implemented** by Conpot template.

Write_register () – helps to perform Modbus write (but in the actual scenario, Register “Value – 1234” write attempt got failed, due to illegal data address)

Function code **134** = Exception response to Function Code **6**

Exception code 2 - indicates requested **register address does not exist or not implemented** by Conpot template.

Read_coils () – helps to read output (of Ex. Pump status, Motor ON/OFF state, Relay output, Actuator output and etc.) (but in the actual scenario it got failed)

Function code **129** = Exception response to Function Code **1**

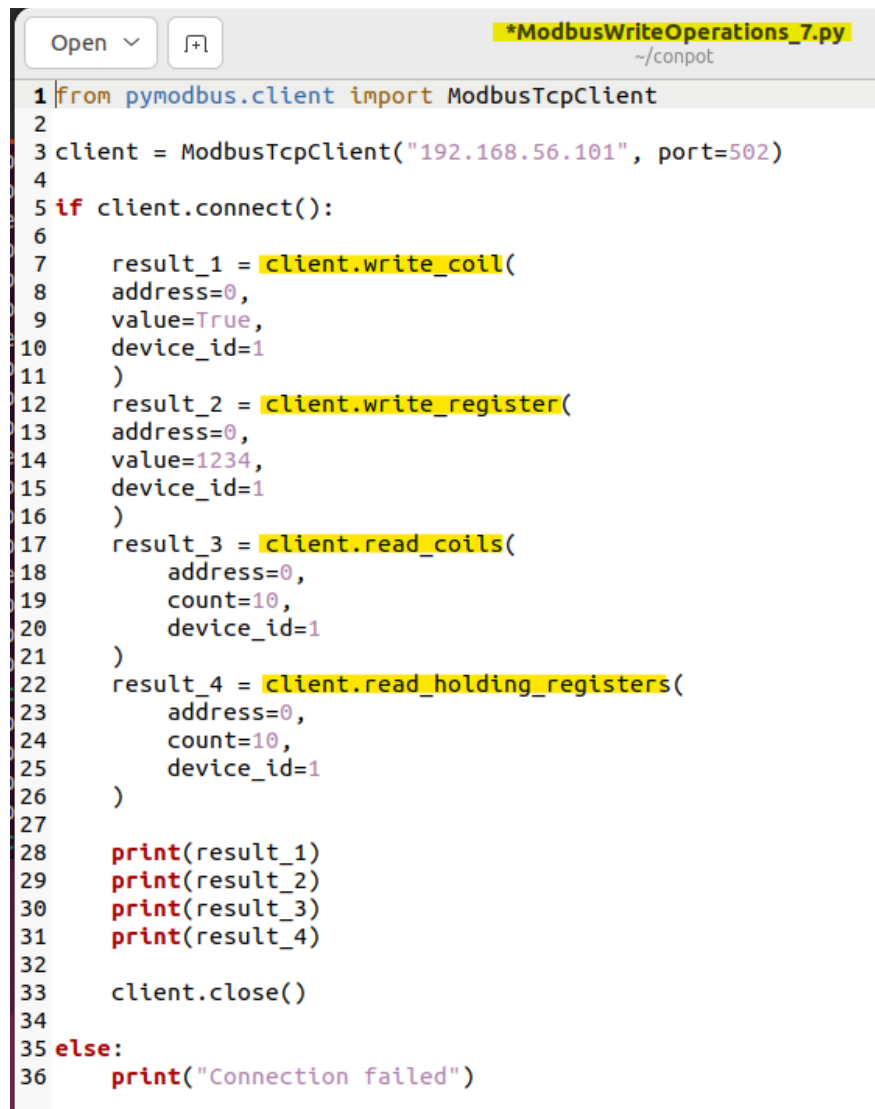
Exception code 2 (illegal data address) indicates requested coil address are not available in the Conpot Modbus configuration.

Read_holding_registers() – Helps holding registers (typically Ex. Process variables, Controller settings, Configuration parameters, Setpoints, Counters etc.)

Function code **131** = Exception response to Function Code **3**

Modbus server received the request but reported that the requested register addresses are invalid or not implemented.

Refer below snapshots of Python script performs following write & read operations.



```
1 from pymodbus.client import ModbusTcpClient
2
3 client = ModbusTcpClient("192.168.56.101", port=502)
4
5 if client.connect():
6
7     result_1 = client.write_coil(
8         address=0,
9         value=True,
10        device_id=1
11    )
12    result_2 = client.write_register(
13        address=0,
14        value=1234,
15        device_id=1
16    )
17    result_3 = client.read_coils(
18        address=0,
19        count=10,
20        device_id=1
21    )
22    result_4 = client.read_holding_registers(
23        address=0,
24        count=10,
25        device_id=1
26    )
27
28    print(result_1)
29    print(result_2)
30    print(result_3)
31    print(result_4)
32
33    client.close()
34
35 else:
36     print("Connection failed")
```

Packet tool evidence:

Write attempt generated Modbus requests reached Conpot server, and feedback is captured in the below screenshot.

Conpot server log shows Modbus server processed incoming requests and generated protocol-compliant exception responses. Obviously says that communication via Modbus service is successful, but the actual requested address found invalid.

```
sivachandran@VirtualBox:~/conpot$ python3 ModbusWriteOperations 7.py
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
sivachandran@VirtualBox:~/conpot$
```

```
sivachandran@VirtualBox:~/conpot$ sudo tcpdump -i lo port 502 -w modbus_capture.pcap
[sudo] password for sivachandran:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C7 packets captured
14 packets received by filter
0 packets dropped by kernel
sivachandran@VirtualBox:~/conpot$
```

Even though **write attempt unsuccessful**, Modbus write operations permitted without appropriate network controls, **attacker could access engineering work, establish TCP connection** and **reachable** to the Modbus service, and communicate via valid Modbus protocol messages, and then tried to actively write requests accepted at the protocol level, ultimately it helps **receive legitimate Modbus exception responses from server** (which could potentially influence industrial processes).

In the original industrial control system, similar Modbus write operations could modify the operational parameters (Pump start/stop commands, Valve open/close status, Motor control signals, Alarm acknowledgement bits, Process setpoints, Temperature or Pressure thresholds, Conveyor control, Actuator states and etc.)

This practice highlights the importance of restricting access to industrial protocols, as successful write operations against a real PLC could alter physical process behaviour.

Part – F: Packet Capture and Protocol Evidence

Object is to capture and analyze Modbus/TCP traffic exchange happened between attacker and Conpot OT/SCADA simulator. Packet analysis used to identify the Modbus Application Protocol (MBAP) header fields, interpret the Modbus function codes.

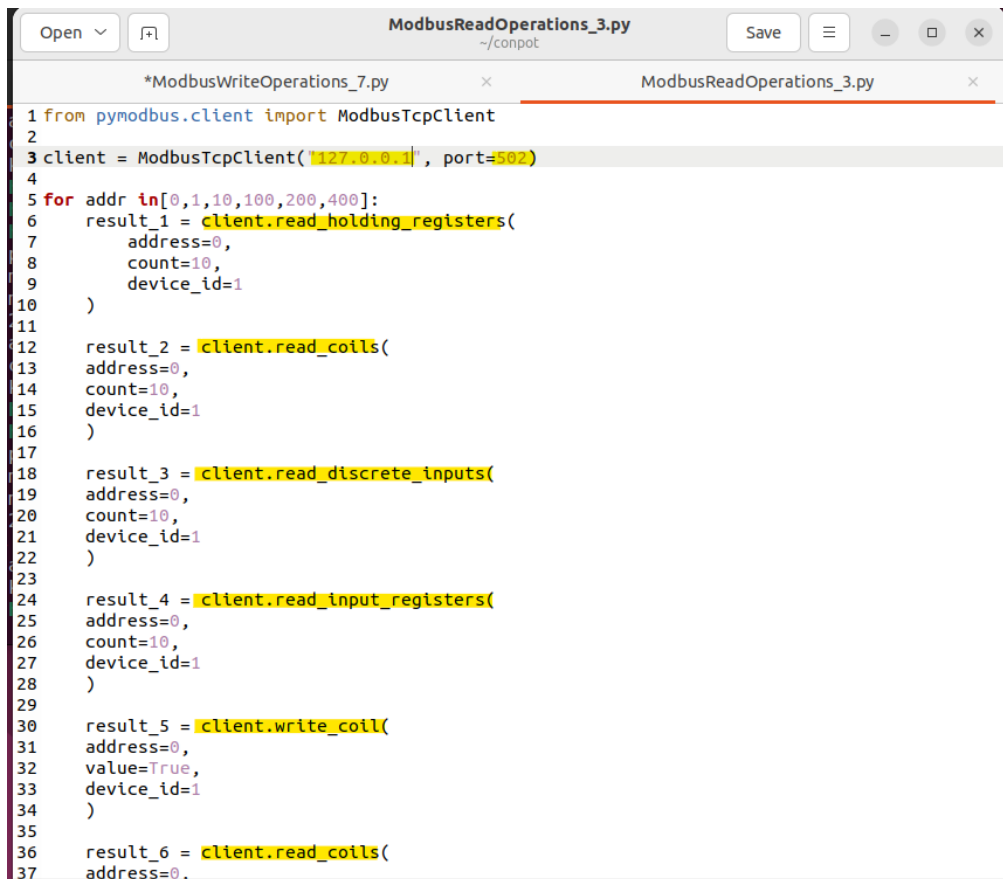
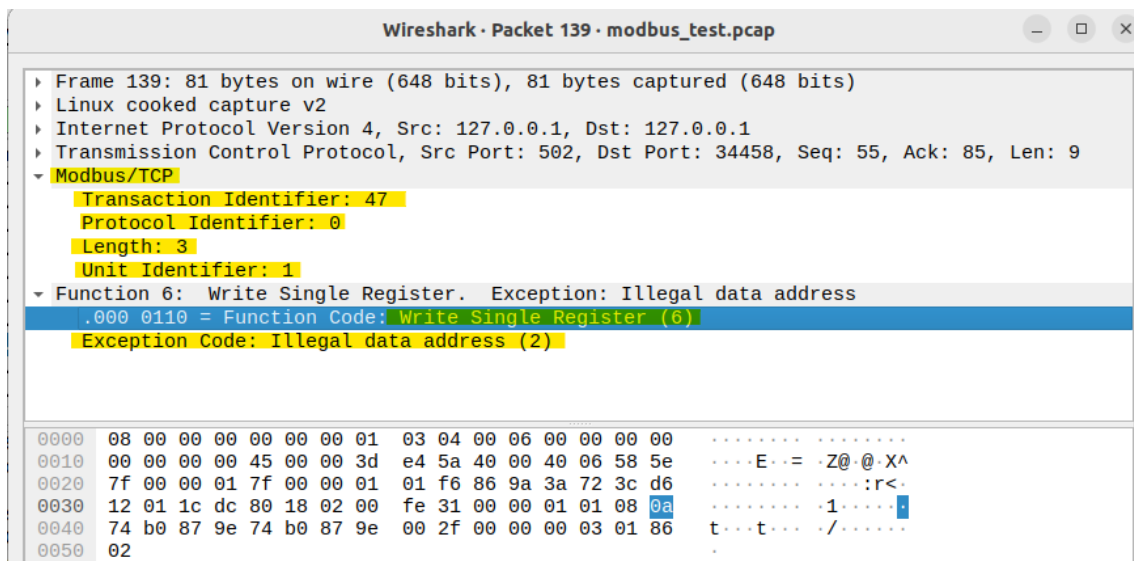
During packet capture Modbus client connected via the local loopback interface (127.0.0.1) which forwarded requests to the docker-hosted Modbus service on TCP port 502. Although the traffic used the loopback interface rather than the host-only interface (192.168.56.101), the captured Modbus/TCP packets are protocol-identified and accurately demonstrate the Modbus request/response exchange required.

In this exercise, Docker port forwarding caused the communication to occur locally via 127.0.0.1 while preserving the Modbus protocol semantics.

Hope this gives evidence that Client communicates with Conpot Modbus service, protocol fields were captured.

```
sivachandran@VirtualBox:~/conpot$ sudo tcpdump -i any -nn port 502 -w modbus_test.pcap
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C144 packets captured
288 packets received by filter
0 packets dropped by kernel
sivachandran@VirtualBox:~/conpot$ wireshark modbus_test.pcap
```

Taken here one sample read attempted to demonstrate:



```

38     count=1,
39     device_id=1
40 )
41
42 result_7 = client.write_register(
43     address=0,
44     value=1234,
45     device_id=1
46 )
47
48 result_8 = client.read_holding_registers(
49     address=0,
50     count=1,
51     device_id=1
52 )
53
54 print("Address:", addr)
55 print(result_1)
56 print(result_2)
57 print(result_3)
58 print(result_4)
59 print(result_5)
60 print(result_6)
61 print(result_7)
62 print(result_8)
63
64 client.close()

```

Python 2 ▾ Tab Width

```

sivachandran@VirtualBox:~/conpot$ python3 ModbusReadOperations_3.py
Address: 0
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
Address: 1
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
Address: 10
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)

```

```

Address: 100
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
Address: 200
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
Address: 400
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=130, exception_code=2)
ExceptionResponse(dev_id=1, function_code=132, exception_code=2)
ExceptionResponse(dev_id=1, function_code=133, exception_code=2)
ExceptionResponse(dev_id=1, function_code=129, exception_code=2)
ExceptionResponse(dev_id=1, function_code=134, exception_code=2)
ExceptionResponse(dev_id=1, function_code=131, exception_code=2)
sivachandran@VirtualBox: ~/conpots

```

Part – G: Security Impact and Defenses

Why is exposing Modbus/TCP to an engineering subnet dangerous?

Unlike modern IT protocols, Modbus/TCP provides neither authentication nor encryption. Which helps any host with TCP port 502 connectivity can send valid Modbus requests (reading coils, discrete inputs, holding registers, and input registers successfully reached the server.

Attacker could access engineering subnet could Read sensor values from PLC registers, Identification of current industrial processes, Enumeration of device capabilities, Process variable read and configuration setting changes, and modification of values potentially lead to modify the device behaviour.

What would change if the same Modbus endpoint were reachable from an enterprise workstation?

Enterprise work stations are compromised through phishing, Malware or ransomware and it may help attacker to (compromise Modbus/TCP) reachable without network restrictions.

Direct attack path from IT environment into industrial control system may get created

and it could impact the following:

Read process information

Identify PLC memory layouts

Updating the control registers, change Actuator states,

Altering process setpoints

Interface with industrial operations

What network control would prevent unauthorized Modbus access?

Restricting TCP port 502 with industrial network segmentation and access control lists (ACLs), it means access only connections from authorized SCADA server and approved engineering workstations.

What monitoring rule would detect suspicious Modbus writes?

Inspection of Modbus function codes would help to detect such access (instead of relying only on IP address or port numbers), and creation of alert whenever write operation happens will help in general.

Ex. "Alert on Function Code 5 (Write Single Coil) etc.

Apart from this, when unauthorized host communicates with TCP port 502 alert should be made by applying network scanning against industrial ports.

Monitor abnormal increases in Modbus write frequency would help as well.

Why is protocol-valid traffic still dangerous?

This assignment has Modbus packets may be completely protocol-compliant and valid but still malicious. (Transaction Identifier, Protocol identifier, Unit identifier, Function code & Register address)

Correctly formatted Modbus packet is not necessarily an authorized packet.

In contrast with IT protocols, Modbus provides no mechanism to verify whether the sender is authorised to perform the requested operation or not.

How should vendor or engineering access be constrained?

Making design with the consideration of Engineering should never access from the enterprise network directly.

Need to introduce the controls like,

- VPN authentication before accessing OT environment
- Multifactor authentication for remote engineers
- Time-limited vendor access approvals
- Vendor account disable when maintenance get completed
- Session authorization and activity log capture
- Dedicated jump host for all engineering sessions

Propose at least three concrete defenses:

Defense mechanism 1 – Allowlist TCP port 502 access:

Configuring industrial firewalls, helps authorized SCADA servers and engineering jump hosts could connect with TCP port 502.

By this way Attacker via other systems sending Modbus commands to PLCS will get prevented.

Defense mechanism 2 – Restricting Modbus to communicate with SCADA server:

Restriction of accepting Modbus requests from designated SCADA server.

During normal operation rather than engineering workstations communicate directly, approval management system should be introduced.

Defense mechanism 3 – Monitoring Modbus Function code requests:

Inspection of Modbus traffic via industrial IDS or network monitoring platform.

(alert for Function code – 5/6/15/16 with respect to Write Single coil, Write Single Register, Write Multiple coils, Write Multiple Registers)

Defense mechanism 4 – Separation of Enterprise IT and OT networks:

Consideration of Engineering workstations should never access with PLCs directly.

Residing in a dedicated OT network protected by industrial firewalls and network segmentation.

Defense mechanism 5 – Arranging dedicated Engineering Jump Host:

Entire engineering and vendor access maintenance need to pass via monitored jump host.

This jump host to provide access approval, user authentication, session recording, session time limit etc.